PhD Ocean Analysis Package

First Draft User Guide and Function Reference

Paulo Victor Silva

July 21, 2026

Abstract

This document describes the first draft of the `phd-ocean-analysis` Python package. The current package focuses on exploratory data analysis (EDA) for oceanographic and atmospheric gridded datasets. The main capabilities include loading multiple NetCDF files, longitude transformation, regional selection, one-dimensional yearly statistics, two-dimensional spatial statistics, seasonal and monthly maps, Albers/Cartopy plotting, and first draft vector diagnostics.

Contents

1 Repository Structure

The recommended repository structure is:

```
phd-ocean-analysis/
          pyproject.toml
          environment.yml
          README.md
          scripts/
                examples/
                    eda/
                        inspect_datasets.py
                        run_jra3q_temperature_eda.py
          src/
                phd_ocean/
                    __init__.py
                    eda/
                        __init__.py
                        io.py
                        stats.py
                        vector.py
                        plotting.py
                        eda_1d.py
                        eda_2d.py
          outputs/
              figures/
              tables/
              netcdf/
```

The package currently follows a `src/` layout. The installable package is located at:

```
src/phd_ocean/
```

The EDA tools are located at:

```
src/phd_ocean/eda/
```

2 Installation

Create or update the Conda environment:

```
cd /home/paulo/Documents/codes/mine/PhD_codes/python/phd-ocean-analysis
conda env update -f environment.yml --prune
conda activate phd-ocean
python -m pip install -e .
```

The editable install is important because it allows changes inside `src/phd_ocean/` to be used without reinstalling the package after every edit. When using Spyder, restart the kernel after changing package source files.

3 Main EDA Workflow

The package is designed around a simple workflow:

1. Load one or many NetCDF files.
2. Standardize longitudes if needed.

3. Select a geographic region.
4. Run one-dimensional EDA for yearly distributions and summary tables.
5. Run two-dimensional EDA for spatial statistics and period maps.
6. Save figures, tables, and NetCDF outputs.

A typical script imports:

```
from phd_ocean.eda import (
    load_many_datasets,
    transform_longitude,
    select_region,
    run_eda_1d,
    run_eda_2d,
)
```

4 Input/Output Utilities: `io.py`

The `io.py` module contains file-loading, coordinate, and spatial-selection utilities.

4.1 Important Functions

Function	Purpose
`expand_paths()`	Expands file paths, lists of paths, and glob patterns into a sorted list of files.
`load_dataset()`	Loads one NetCDF file or one group of files into an `xarray.Dataset`.
`load_many_datasets()`	Loads many NetCDF files, optionally applying preprocessing transformations before concatenating.
`infer_time_dim()`	Attempts to identify the time dimension from common names such as `time`, `TIME`, or related variants.
`standardize_time_dim()`	Renames the detected time coordinate to a standard name.
`resample_time()`	Resamples a dataset or data array in time, for example monthly or yearly averages.
`transform_longitude()`	Transforms longitudes from 0–360 degrees to -180–180 degrees and sorts by longitude.
`select_region()`	Selects a geographic box using minimum and maximum longitude and latitude.
`select_vertical_section_ij()`	Selects a vertical section from model grids using `i/j` indices. Useful for NEMO-like grids.
`make_section_indices()`	Creates a list of section indices between two grid points.

4.2 Example: Load Many Files and Select a Region

```
from glob import glob

from phd_ocean.eda import (
    load_many_datasets,
    transform_longitude,
    select_region,
```

```
)

DATA_PATHS = "/home/paulo/Documents/data/atmospheric/JRA3Q/"
DATA_FILES = sorted(glob(DATA_PATHS + "q2_JRA3Q_y*.nc"))

def fix_longitude(ds):
    return transform_longitude(ds, lon_name="lon")

def cut_region(ds):
    return select_region(
        ds,
        lat_min=45,
        lat_max=72,
        lon_min=-105,
        lon_max=-60,
        lat_name="lat",
        lon_name="lon",
    )

ds = load_many_datasets(
    DATA_FILES,
    concat_dim="time",
    transform=fix_longitude,
    transform_second=cut_region,
)
```

5 Statistics Utilities: `stats.py`

The `stats.py` module contains scalar, circular, vector, and grouped statistics.

5.1 Scalar Statistics

The current scalar statistics include:

- mean
- standard deviation
- median or q50
- interquartile range
- Yule-Kendall index
- skewness
- kurtosis
- minimum and maximum
- selected percentiles

5.2 Important Functions

Function	Purpose
`compute_1d_statistics()`	Computes summary statistics from a one-dimensional array or flattened data.
`compute_1d_statistics_by_year()`	Groups a time series by year and computes yearly statistics.
`compute_2d_statistics()`	Computes spatial maps of statistics along a chosen time dimension.
`aggregate_periods()`	Aggregates data into seasonal, monthly, or annual means or other reductions.
`circular_mean_direction()`	Computes circular mean direction for directional data.
`circular_std_direction()`	Computes circular standard deviation for directional data.
`compute_1d_vector_statistics()`	Computes statistics for paired `u`/`v` vector data.
`compute_1d_vector_statistics_by_year()`	Computes yearly vector statistics, including speed and direction diagnostics.

5.3 Example: 2D Statistics

```
from phd_ocean.eda import compute_2d_statistics

stats_maps = compute_2d_statistics(
    ds["q2m"],
    time_dim="time",
    statistics=["mean", "std", "q50", "iqr", "yki", "kurtosis"],
)
```

6 Vector Utilities: `vector.py`

The `vector.py` module handles paired vector components.

6.1 Important Functions

Function	Purpose
`compute_speed()`	Computes vector speed from `u` and `v`.
`compute_direction()`	Computes vector direction using a selected convention.
`compute_vector_diagnostics()`	Returns a dataset containing `u`, `v`, speed, and direction.
`compute_vector_diagnostics_from_dataset()`	Computes vector diagnostics directly from a dataset and variable names.
`decompose_speed_direction()`	Converts speed and direction back into vector components.

6.2 Direction Conventions

The package supports different direction conventions:

- `math`: 0 degrees points east and 90 degrees points north.
- `bearing`: 0 degrees points north and 90 degrees points east.
- `coming_from`: meteorological/oceanographic convention for direction from which flow comes.

6.3 Example: Speed and Direction

```
from phd_ocean.eda import compute_speed, compute_direction

u = ds["u"]
v = ds["v"]

speed = compute_speed(u, v)

direction = compute_direction(
    u,
    v,
    convention="bearing",
    units="degrees",
)
```

7 Plotting Utilities: `plotting.py`

The `plotting.py` module contains map, subplot, violin, and vector plotting functions.

7.1 Important Functions

Function	Purpose
`plot_violin_by_year()`	Creates yearly violin plots from grouped data.
`plot_2d_map()`	Plots a single two-dimensional scalar field on a Cartopy map.
`plot_2d_subplots()`	Plots grouped fields, such as seasonal or monthly maps, as subplots.
`plot_quiver()`	Adds vector arrows to an existing Cartopy axis.
`plot_scalar_with_vectors()`	Plots a scalar background field with vector arrows on top.
`plot_vector_map()`	Plots vector speed as the background field and vector arrows on top.
`plot_scalar_vector_subplots()`	Plots grouped scalar fields with grouped vector arrows.

7.2 Cartopy Projection Logic

The plotting functions separate the visual map projection from the coordinate system of the data:

```
projection = ccrs.AlbersEqualArea(
    central_longitude=-80,
    central_latitude=60,
    standard_parallels=(50, 70),
)

data_crs = ccrs.PlateCarree()
```

The recommended interpretation is:

Name	Meaning
`projection`	The projection used to draw the map. Example: Albers Equal Area.
`data_crs`	The coordinate reference system of the input longitude and latitude data. Usually `PlateCarree`.
`gridline_crs`	The coordinate system used to draw lon/lat gridlines. Usually `PlateCarree`.
`extent`	The plotting region in longitude and latitude coordinates.

7.3 Albers Equal Area Example

```
import cartopy.crs as ccrs

projection = ccrs.AlbersEqualArea(
    central_longitude=-80,
    central_latitude=60,
    standard_parallels=(50, 70),
)

plot_kwargs = {
    "projection": projection,
    "data_crs": ccrs.PlateCarree(),
    "extent": [-105, -60, 45, 72],
    "levels": 30,
    "cmap": "viridis",
    "title": None,
    "colorbar_label": "Specific humidity",
    "gridline_step": 5,
    "map_features": {
        "land": False,
        "coastline": True,
        "borders": False,
        "states": False,
        "draw_labels": True,
        "gridline_crs": ccrs.PlateCarree(),
    },
}
```

For atmospheric variables, set:

```
"map_features": {
    "land": False,
    "coastline": True,
}
```

For ocean variables, set:

```
"map_features": {
    "land": True,
    "coastline": True,
}
```

7.4 Current Gridline Note

For Albers Equal Area maps, the package disables Cartopy geographic boundary labels with:

```
gl.geo_labels = False
```

This preserves standard longitude and latitude labels while avoiding failures in the gridliner when the map boundary is projected.

8 One-Dimensional EDA: eda_1d.py

The eda_1d.py module provides high-level workflows for one-dimensional summaries.

8.1 Important Functions

Function	Purpose
select_variable()	Selects a variable from a dataset or validates a data array.
prepare_1d_data()	Prepares a variable for one-dimensional analysis, including optional depth selection.
xarray_to_yearly_dataframe()	Converts xarray data into a yearly dataframe used by table and violin workflows.
save_summary_table()	Saves yearly summary statistics to CSV or Excel.
run_eda_1d()	High-level one-dimensional EDA workflow for scalar variables.
prepare_1d_vector_data()	Prepares paired u/v components for vector EDA.
run_eda_1d_vector()	High-level one-dimensional workflow for vector speed and direction statistics.

8.2 Example: Scalar 1D EDA

```
from phd_ocean.eda import run_eda_1d

summary = run_eda_1d(
    ds,
    variable="q2m",
    time_dim="time",
    depth_index=None,
    output_table="outputs/tables/jra3q_q2m_1d_summary.csv",
    output_violin="outputs/figures/jra3q_q2m_violin.png",
    ylabel="2 m specific humidity",
    title="JRA3Q 2 m specific humidity yearly distribution",
    show=False,
)
```

9 Two-Dimensional EDA: eda_2d.py

The eda_2d.py module provides high-level workflows for spatial statistics and period maps.

9.1 Important Functions

Function	Purpose
`prepare_2d_field()`	Selects a variable, applies optional depth selection, and prepares a 2D or time-varying 2D field.
`run_eda_2d_statistics()`	Computes full-period spatial statistics and optionally plots each statistic.
`run_eda_2d_periods()`	Computes and plots seasonal, monthly, and/or annual aggregated maps.
`run_eda_2d()`	General high-level wrapper for statistics, period maps, or both.

9.2 Example: 2D Statistics

```
from phd_ocean.eda import run_eda_2d

run_eda_2d(
    ds,
    variable="q2m",
    time_dim="time",
    depth_index=None,
    mode="statistics",
    statistics=["mean", "std", "q50", "iqr", "yki", "kurtosis"],
    output_dir="outputs/figures/jra3q_specific_humidity/statistics",
    output_statistics_nc="outputs/netcdf/
       jra3q_specific_humidity_statistics.nc",
    lon_name="lon",
    lat_name="lat",
    plot_kwargs=plot_kwargs,
    show=False,
)
```

9.3 Example: Seasonal and Monthly Maps

```
run_eda_2d(
    ds,
    variable="q2m",
    time_dim="time",
    depth_index=None,
    mode="periods",
    periods=["seasonal", "monthly"],
    reducer="mean",
    output_dir="outputs/figures/jra3q_specific_humidity/periods",
    save_periods_netcdf=True,
    lon_name="lon",
    lat_name="lat",
    plot_kwargs=plot_kwargs,
    show=False,
)
```

For first tests, avoid annual maps when the dataset spans many decades, because this may create a very large yearly subplot figure.

10 Complete Atmospheric Example

```
from glob import glob
from pathlib import Path

import cartopy.crs as ccrs

from phd_ocean.eda import (
    load_many_datasets,
    transform_longitude,
    select_region,
    run_eda_1d,
    run_eda_2d,
)

PROJECT_DIR = Path(
    "/home/paulo/Documents/codes/mine/PhD_codes/python/phd-ocean-
       analysis"
)

DATA_PATHS = "/home/paulo/Documents/data/atmospheric/JRA3Q/"
DATA_FILES = sorted(glob(DATA_PATHS + "q2_JRA3Q_y*.nc"))

VAR_NAME = "q2m"
TIME_DIM = "time"

REGION = {
    "lat_min": 45,
    "lat_max": 72,
    "lon_min": -105,
    "lon_max": -60,
}

EXTENT = [
    REGION["lon_min"],
    REGION["lon_max"],
    REGION["lat_min"],
    REGION["lat_max"],
]

def fix_longitude(ds):
    return transform_longitude(ds, lon_name="lon")

def cut_region(ds):
    return select_region(
        ds,
        lat_min=REGION["lat_min"],
        lat_max=REGION["lat_max"],
        lon_min=REGION["lon_min"],
        lon_max=REGION["lon_max"],
        lat_name="lat",
        lon_name="lon",
    )

projection = ccrs.AlbersEqualArea(
    central_longitude=-80,
    central_latitude=60,
    standard_parallels=(50, 70),
)
```

```
plot_kwargs = {
    "projection": projection,
    "data_crs": ccrs.PlateCarree(),
    "extent": EXTENT,
    "levels": 30,
    "cmap": "viridis",
    "title": None,
    "colorbar_label": "Specific humidity",
    "gridline_step": 5,
    "map_features": {
        "land": False,
        "coastline": True,
        "borders": False,
        "states": False,
        "draw_labels": True,
        "gridline_crs": ccrs.PlateCarree(),
    },
}

ds = load_many_datasets(
    DATA_FILES,
    concat_dim=TIME_DIM,
    transform=fix_longitude,
    transform_second=cut_region,
)

summary = run_eda_1d(
    ds,
    variable=VAR_NAME,
    time_dim=TIME_DIM,
    depth_index=None,
    output_table=PROJECT_DIR / "outputs/tables/jra3q_q2m_1d_summary.csv
        ",
    output_violin=PROJECT_DIR / "outputs/figures/jra3q_q2m_violin.png",
    ylabel="2 m specific humidity",
    title="JRA3Q 2 m specific humidity yearly distribution",
    show=False,
)

run_eda_2d(
    ds,
    variable=VAR_NAME,
    time_dim=TIME_DIM,
    depth_index=None,
    mode="statistics",
    statistics=["mean", "std", "q50", "iqr", "yki", "kurtosis"],
    output_dir=PROJECT_DIR / "outputs/figures/jra3q_specific_humidity/
        statistics",
    output_statistics_nc=PROJECT_DIR / "outputs/netcdf/
        jra3q_q2m_statistics.nc",
    lon_name="lon",
    lat_name="lat",
    plot_kwargs=plot_kwargs,
    show=False,
)

run_eda_2d(
```

```
    ds,
    variable=VAR_NAME,
    time_dim=TIME_DIM,
    depth_index=None,
    mode="periods",
    periods=["seasonal", "monthly"],
    reducer="mean",
    output_dir=PROJECT_DIR / "outputs/figures/jra3q_specific_humidity/
        periods",
    save_periods_netcdf=True,
    lon_name="lon",
    lat_name="lat",
    plot_kwargs=plot_kwargs,
    show=False,
)
```

11 Current Development Roadmap

The next planned development steps are:

1. Improve colorbar handling:
    - label size;
    - tick size;
    - scientific notation;
    - custom tick formatters;
    - vertical and horizontal colorbar options;
    - shared colorbar control for subplots.
2. Improve colormap handling:
    - sequential colormaps for positive variables;
    - diverging colormaps for anomalies and signed variables;
    - `cmocean` support;
    - custom level arrays.
3. Apply the scalar EDA workflow to real atmospheric datasets.
4. Test vector workflows:
    - load paired `u`/`v` fields;
    - compute speed and direction;
    - compute circular statistics;
    - plot vector maps with quiver arrows;
    - test seasonal and monthly vector subplots.
5. Add package tests:
    - unit tests for statistics;
    - unit tests for longitude transformation and regional selection;
    - tests for vector speed and direction;
    - smoke tests for plotting functions.

12 Git Workflow

A simple Git workflow for this package is:

```
git status
git checkout -b feature-name
git add src scripts README.md environment.yml pyproject.toml
git commit -m "Describe the change"
git push -u origin feature-name
```

To merge into `main` locally:

```
git checkout main
git pull origin main
git merge feature-name
git push origin main
```

13 Notes

This document describes the current first draft of the package. The public API is still evolving. The guide should be updated after each major refactor, especially after the colorbar, colormap, real-data, vector, and testing improvements.